Chapter 4
Software Development Process Models

➢ Introduction

Software development models are various processes or methods that are chosen for project development depending on the objectives and goals of the project. Many development life cycle models have been developed to achieve various essential objectives. Models specify the various steps of the process and the order in which they are executed.

What is Software Modeling?
Software Modeling is the process of creating abstract representations of a software system. These models serve as blueprints that guide developers, designers, and stakeholders through the system's structure, behavior, and functionality.
By using diagrams and various modeling languages, software modeling helps in visualizing and understanding the complex aspects of the software, making it easier to plan, develop, and manage the system.

➢ Waterfall Model

The **Waterfall Model** is a **Traditional Software Development Methodology**. It was first introduced by **Winston W. Royce** in 1970. It is a linear and sequential approach to software development that consists of several phases.
This classical waterfall model is simple and idealistic. It is important because most other **Types of Software Development Life Cycle Models** are a derivative of this.

What is the SDLC Waterfall Model?
The waterfall model is a Software Development Model used in the context of large, complex projects, typically in the field of information technology. It is characterized by a structured, sequential approach to **Project Management** and **Software Development**.
The Waterfall Model is useful in situations where the project requirements are well-defined and the project goals are clear. It is often used for large-scale projects with long timelines, where there is little room for error and the project stakeholders need to have a high level of confidence in the outcome.

Phases of Waterfall Model
Waterfall Model divides the life cycle into a set of phases. The development process can be considered as a sequential flow in the waterfall. The different sequential phases of the classical waterfall model are follow:

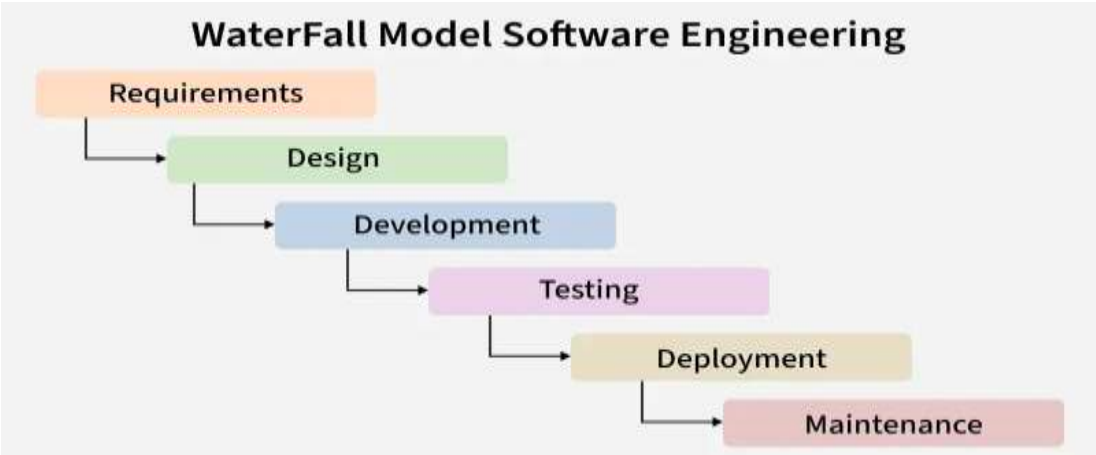


Fig : Waterfall Model-Software Engineering

Phases:

1. **Requirements Analysis and Specification**

Requirement Analysis and specification phase aims to understand the exact requirements of the customer and document them properly. This phase consists of two different activities.

i. Requirement Gathering and Analysis: Firstly, all the requirements regarding the software are gathered from the customer and then the gathered requirements are analyzed. The goal of the analysis part is to remove incompleteness (an incomplete requirement is one in which some parts of the actual requirements have been omitted) and inconsistencies (an inconsistent requirement is one in which some part of the requirement contradicts some other part).

ii. Requirement Specification: These analyzed requirements are documented in a software requirement specification (SRS) document. SRS document serves as a contract between the development team and customers. Any future dispute between the customers and the developers can be settled by examining the SRS document.

2. Design

The goal of this **Software Design Phase** is to convert the requirements acquired in the SRS into a format that can be coded in a programming language. It includes high-level and detailed design as well as the overall software architecture. A **Software Design Document** is used to document all of this effort (SDD).

- **High-Level Design (HLD)**: This phase focuses on outlining the broad structure of the system. **It highlights the key components and how they interact with each other**, giving a clear overview of the system's architecture.
- **Low-Level Design (LLD)**: Once the high-level design is in place, this phase zooms into the details. **It breaks down each component into smaller parts and provides specifics about how each part will function,** guiding the actual coding process.

3. **Development**

In the **Development Phase** software design is translated into source code using any suitable programming language. Thus, each designed module is coded. The unit testing phase aims to check whether each module is working properly or not.

- In this phase, developers begin writing the actual source code based on the designs created earlier.

- The goal is to transform the design into working code using the most suitable programming languages.
- Unit tests are often performed during this phase to make sure that each component functions correctly on its own.

4. **Testing and Deployment**

i. Testing: Integration of different modules is undertaken soon after they have been coded and unit tested. Integration of various modules is carried out incrementally over several steps. During each integration step, previously planned modules are added to the partially integrated system and the resultant system is tested. Finally, after all the modules have been successfully integrated and tested, the full working system is obtained and system testing is carried out on this. System testing consists of three different kinds of testing activities as described below.

- **Alpha testing:** Alpha testing is the system testing performed by the **development team**.
- **Beta testing:** Beta testing is the system testing performed by a friendly set of **customers.**
- **Acceptance testing:** After the software has been delivered, the customer performs acceptance testing to determine **whether to accept the delivered software or reject it.**

ii. Deployment: Once the software has been thoroughly tested, it's time to deploy it to the customer or end-users. This means making the software ready and available for use, often by moving it to a live or staging environment.

During this phase, we also focus on helping users get comfortable with the software by providing training, setting up necessary environments, and ensuring everything is running smoothly. The goal is to make sure the system works as expected in real-world conditions and that users can start using it without any hitches.

5. **Maintenance**

In **Maintenance Phase** is the most important phase of a software life cycle. The effort spent on maintenance is 60% of the total effort spent to develop a full software. There are three types of maintenance.

- **Corrective Maintenance:** This type of maintenance is **carried out to correct errors** that were not discovered during the product development phase.
- **Perfective Maintenance:** This type of maintenance is carried out **to enhance the functionalities of the system based on the customer's request.**
- **Adaptive Maintenance:** Adaptive maintenance is usually required for porting the software **to work in a new environment such as working on a new computer platform or with a new operating system**.

Advantages of Waterfall Model

The classical waterfall model is an idealistic model for software development. It is very simple, so it can be considered the basis for other software development life cycle models. Below are some of the major advantages of this SDLC model.

- **Easy to Understand:** The Waterfall Model is very simple and easy to understand.
- **Individual Processing:** Phases in the Waterfall model are processed one at a time.
- **Properly Defined:** In the waterfall model, each stage in the model is clearly defined.

- **Clear Milestones:** The Waterfall model has very clear and well-understood milestones.
- **Properly Documented:** Processes, actions, and results are very well documented.
- **Reinforces Good Habits:** The Waterfall Model reinforces good habits like define-before-design and design-before-code.
- **Working:** Waterfall Model works well for smaller projects and projects where requirements are well understood.

Disadvantages of Waterfall Model

The Classical Waterfall Model suffers from various shortcomings we can't use it in real projects, but we use other software development lifecycle models which are based on the classical waterfall model. Below are some major drawbacks of this model.

- **No Feedback Path:** In the classical waterfall model evolution of software from one phase to another phase is like a waterfall. It assumes that no error is ever committed by developers during any phase. Therefore, it does **not incorporate any mechanism for error correction.**
- **Difficult to accommodate Change Requests:** This model assumes that all the customer requirements can be completely and correctly defined at the beginning of the project, but the customer's requirements keep on changing with time. It is **difficult to accommodate any change requests** after the requirements specification phase is complete.
- **No Overlapping of Phases:** This model recommends that a new phase can start only after the completion of the previous phase. But in real projects, this can't be maintained. **To increase efficiency and reduce cost, phases may overlap.**
- **Limited Flexibility:** The Waterfall Model is a rigid and linear approach to software development, which means that it is **not well-suited for projects with changing or uncertain requirements**. Once a phase has been completed, it is difficult to make changes or go back to a previous phase.
- **Limited Stakeholder Involvement:** The Waterfall Model is a structured and sequential approach, which means that stakeholders are typically involved in the early phases of the project (**requirements gathering and analysis**) but may not be involved in the later phases **(implementation, testing, and deployment).**
- **Late Defect Detection:** In the Waterfall Model, testing is typically done toward the end of the development process. This means that **defects may not be discovered until late in the development process, which can be expensive and time-consuming to fix.**
- **Lengthy Development Cycle:** The Waterfall Model can result in **a lengthy development cycle,** as each phase must be completed before moving on to the next. This can result in delays and increased costs if requirements change or new issues arise..

- **Incremental Model**

The Incremental model is a software Development approach which is used to breakdown the project into smaller and easily manageable parts. In these, each part passes through Requirement, Design, Testing phases and Implementation phase. The overall process continues until we got the complete System.

Incremental Process Model

The Incremental Process Model is a method of software development where the system is built step by step. Instead of delivering the whole system at once, it is developed and delivered in small parts called increments. Each increment builds upon the previous one by adding new functionality, until the complete system is finished.

Phases of the Incremental Model

The phases of Incremental model are divided into the four parts which is Requirement, Design, Testing phases and Implementation phase. In those phases, the process continues until we got the expected output at the end.

The Incremental model is a software Development approach which is used to breakdown the project into smaller and easily manageable parts. In these, each part passes through Requirement, Design, Testing phases and Implementation phase. The overall process continue until we got the complete System.

Phases of the Incremental Model

The phases of Incremental model is divided into the four parts which is Requirement, Design, Testing phases and Implementation phase. In those phase, the process continues until we got the expected output at the end.

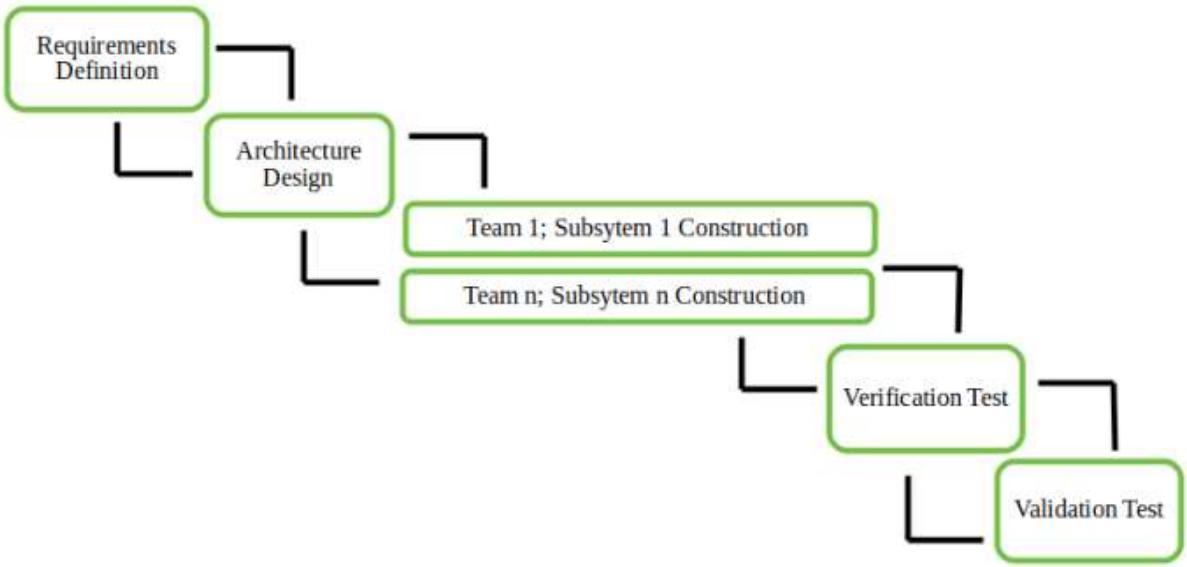


Fig: Phases of incremental model

- **Requirement Analysis**

The first step in the Incremental Model is understanding what the software needs to do. The team gathers the requirements from the product experts and clearly defines the system's functional needs. This phase is important because it sets the foundation for everything else in the development process.

- **Design & Development**

Next, the team focuses on designing how the software will function and starts developing it. They work on adding new features and making sure the system works as expected. The design and development steps go hand-in-hand to build the functionality of the software.

- **Testing**

Once a feature is developed, it goes through testing. The testing phase checks how the software performs, including both new and existing features. The team uses different testing methods to make sure everything is working correctly.

- **Implementation**

This phase involves writing the final code based on the design and development steps. After testing the functionality, the team verify that everything is working as planned. By the end of this phase, the product is gradually improved and updated until it becomes the final working version.

Advantages of Incremental Process Model

The Incremental Model of software development builds a system in small, manageable sections (increments), making it a good choice for many projects. Below are the key advantages:

- **Faster Software Delivery**
  - Initial working versions of the software can be delivered quickly.
  - Early delivery increases customer satisfaction and feedback opportunities.
- **Clear Understanding for Clients**
  - Clients get to see parts of the system at each stage.
  - This visibility ensures that the final product meets their expectations.
- **Easy to Implement Changes**
  - Requirements can evolve, and changes can be incorporated in subsequent increments.
  - It supports flexibility without heavily disrupting earlier stages.
- **Effective Risk Management**
  - Risks can be identified and handled early due to the staged approach.
  - Each increment allows for testing and validation, reducing the impact of unforeseen issues.
- **Flexible Criteria and Scope**
  - Requirements can be adjusted without a major cost increase.
  - Better scope management helps keep the project aligned with business goals.
- **Cost-Effective**
  - Compared to models like the Waterfall, the incremental model is generally more cost-efficient.

- Budget is spread across stages, making it easier to manage finances.

- **Simpler Error Identification**
  - Since development is done in parts, it's easier to pinpoint and fix errors within a specific increment.
  - Testing each module separately enhances quality and reliability.

Disadvantages of Incremental Process Model

Incremental Model comes with some limitations and challenges. Below are the key disadvantages:

- **Requires a Skilled Team and Proper Planning**
  - Successful implementation demands an experienced team.
  - Poor planning or coordination can lead to confusion between increments.
- **Cost Can Increase Over Time**
  - Due to repeated testing, redesign, and integration in every cycle, the overall project cost may rise.
  - Continuous iteration involves added overhead
- **Incomplete Requirement Gathering Can Cause Design Issues**
  - If all requirements are not identified early, the system architecture may not support future needs.
  - Inadequate upfront design can lead to rework and architectural mismatches.
- **Lack of Smooth Flow Between Increments**
  - Each iteration may function independently, which can create inconsistencies.
  - There might be integration challenges when combining all the increments into a unified product.
- **High Effort to Fix Repeated Issues**
  - A defect in one increment may exist in others.
  - Fixing the same issue across multiple units can be time-consuming and resource-intensive.

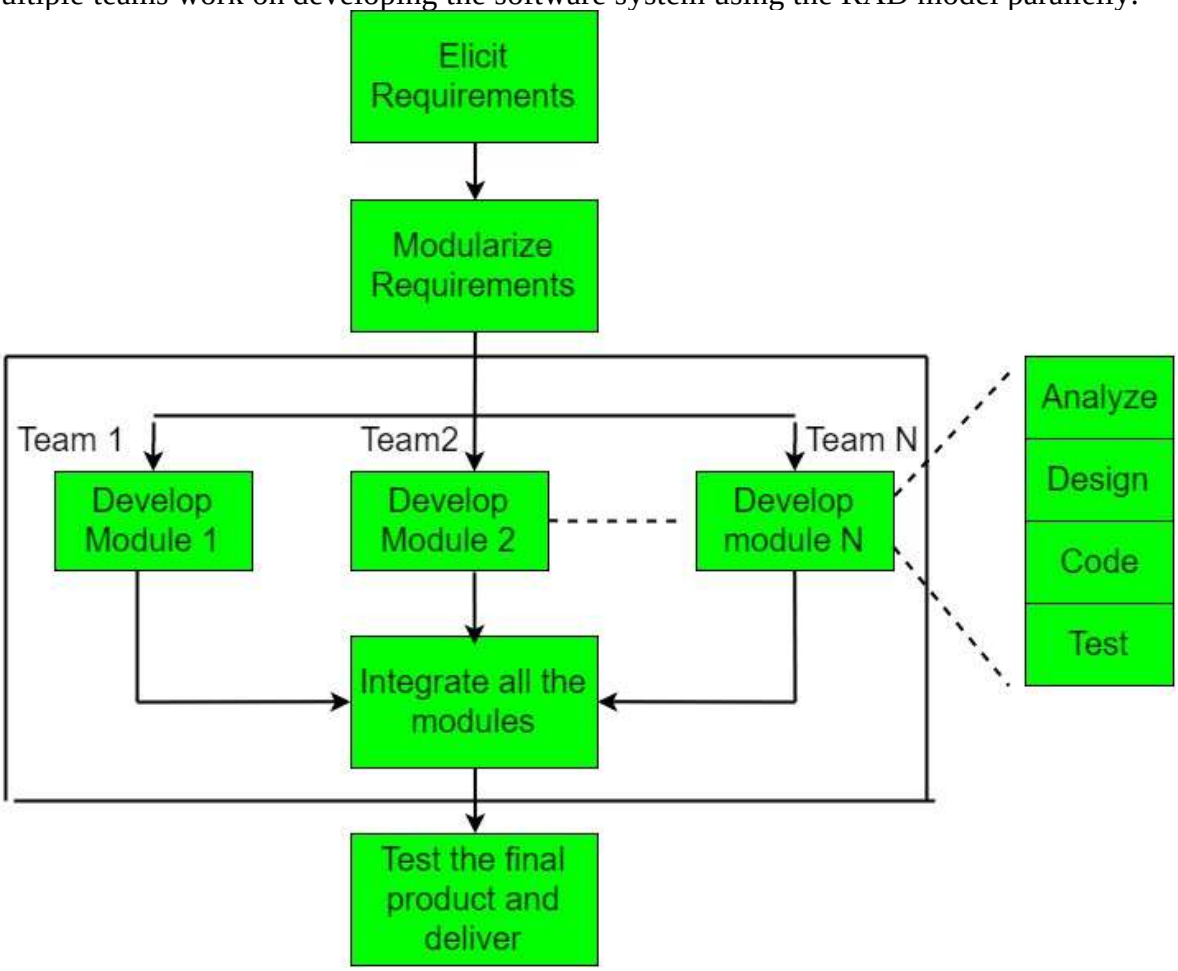
- **Rapid Application Development Model (RAD) Model**

The RAD model or Rapid Application Development model is a type of software development methodology that emphasizes quick and iterative release cycles, primarily focusing on delivering working software in shorter timelines. Unlike traditional models such as the Waterfall model, RAD is designed to be more flexible and responsive to user feedback and changing requirements throughout the development process.

What is RAD Model in Software Engineering?

IBM first proposed the Rapid Application Development or RAD Model in the 1980s. The RAD model is a type of incremental process model in which there is a concise development cycle. The RAD model is used when the requirements are fully understood and the component-based construction approach is adopted. Various phases in RAD are Requirements Gathering, Analysis and **Planning**, **Design**, **Build** or **Construction**, and finally **Deployment**.

The critical feature of this model is the use of powerful development tools and techniques. A software project can be implemented using this model if the project can be broken down into small modules wherein each module can be assigned independently to separate teams. These modules can finally be combined to form the final product. Development of each module involves the various basic steps as in the waterfall model i.e. analyzing, designing, coding, and then testing, etc. as shown in the figure. Another striking feature of this model is a short period i.e. the time frame for delivery(time-box) is generally 60-90 days.
Multiple teams work on developing the software system using the RAD model parallelly.



Rapid application development model (RAD)

The use of powerful developer tools such as JAVA, C++, Visual BASIC, XML, etc. is also an integral part of the projects. This model consists of 4 basic phases:

1. **Requirements Planning -** This involves the use of various techniques used in requirements elicitation like brainstorming, task analysis, form analysis, user scenarios, FAST (Facilitated Application Development Technique), etc. It also consists of the entire structured plan describing the critical data, methods to obtain it, and then processing it to form a final refined model.
2. **User Description -** This phase consists of taking user feedback and building the prototype using developer tools. In other words, it includes re-examination and validation of the data collected in the first phase. The dataset attributes are also identified and elucidated in this phase.
3. **Construction -** In this phase, refinement of the prototype and delivery takes place. It includes the actual use of powerful automated tools to transform

processes and data models into the final working product. All the required modifications and enhancements are to be done in this phase.

4. **Cutover -** All the interfaces between the independent modules developed by separate teams have to be tested properly. The use of powerfully automated tools and subparts makes testing easier. This is followed by acceptance testing by the user.

The process involves building a rapid prototype, delivering it to the customer, and taking feedback. After validation by the customer, the SRS document is developed and the design is finalized.

Objectives of Rapid Application Development Model (RAD)

1. **Speedy Development**
Accelerating the software development process is RAD's main goal. RAD prioritizes rapid prototyping and iterations to produce a working system as soon as possible. This is especially helpful for projects when deadlines must be met.

2**. Adaptability and Flexibility**
RAD places a strong emphasis on adapting quickly to changing needs. Due to the model's flexibility, stakeholders can modify and improve the system in response to changing requirements and user input.

3**. Stakeholder Participation**
Throughout the development cycle, RAD promotes end users and stakeholders' active participation. Collaboration and frequent feedback make it possible to make sure that the changing system satisfies both user and corporate needs.

4. **Improved Interaction**
Development teams and stakeholders may collaborate and communicate more effectively thanks to RAD. Frequent communication and feedback loops guarantee that all project participants are in agreement, which lowers the possibility of misunderstandings.

5. **Improved Quality via Prototyping**
Prototypes enable early system component testing and visualization in Rapid Application Development (RAD). This aids in spotting any problems, confirming design choices, and guaranteeing that the finished product lives up to consumer expectations.

6. **Customer Satisfaction**
Delivering a system that closely satisfies user expectations and needs is the goal of RAD. Through rapid delivery of functioning prototypes and user involvement throughout the development process, Rapid Application Development (RAD) enhances the probability of customer satisfaction with the final product.

Advantages of Rapid Application Development Model (RAD)

- The use of reusable components helps to reduce the cycle time of the project.
- Feedback from the customer is available at the initial stages.
- Reduced costs as fewer developers are required.
- The use of powerful development tools results in better quality products in comparatively shorter periods.

- The progress and development of the project can be measured through the various stages.
- It is easier to accommodate changing requirements due to the short iteration time spans.
- Productivity may be quickly boosted with a lower number of employees.

Disadvantages of Rapid application development model (RAD)

- The use of powerful and efficient tools requires highly skilled professionals.
- The absence of reusable components can lead to the failure of the project.
- The team leader must work closely with the developers and customers to close the project on time.
- The systems which cannot be modularized suitably cannot use this model.
- Customer involvement is required throughout the life cycle.
- It is not meant for small-scale projects as in such cases, the cost of using automated tools and techniques may exceed the entire budget of the project.
- Not every application can be used with RAD.

➢ **Evolutionary process models**

The evolutionary model is a combination of the Iterative and Incremental models of the software development life cycle. Delivering your system in a big bang release, delivering it in incremental process over time is the action done in this model. Some initial requirements and architecture envisioning need to be done. It is better for software products that have their feature sets redefined during development because of user feedback and other factors.

What is the Evolutionary Model?

The Evolutionary development model divides the development cycle into smaller, incremental waterfall models in which users can get access to the product at the end of each cycle.

1. Feedback is provided by the users on the product for the planning stage of the next cycle and the development team responds, often by changing the product, plan, or process.
2. Therefore, the software product evolves with time.
3. All the models have the disadvantage that the duration of time from the start of the project to the delivery time of a solution is very high.
4. The evolutionary model solves this problem with a different approach.
5. The evolutionary model suggests breaking down work into smaller chunks, prioritizing them, and then delivering those chunks to the customer one by one.
6. The number of chunks is huge and is the number of deliveries made to the customer.
7. The main advantage is that the customer's confidence increases as he constantly gets quantifiable goods or services from the beginning of the project to verify and validate his requirements.
8. The model allows for changing requirements as well as all work is broken down into maintainable work chunks.

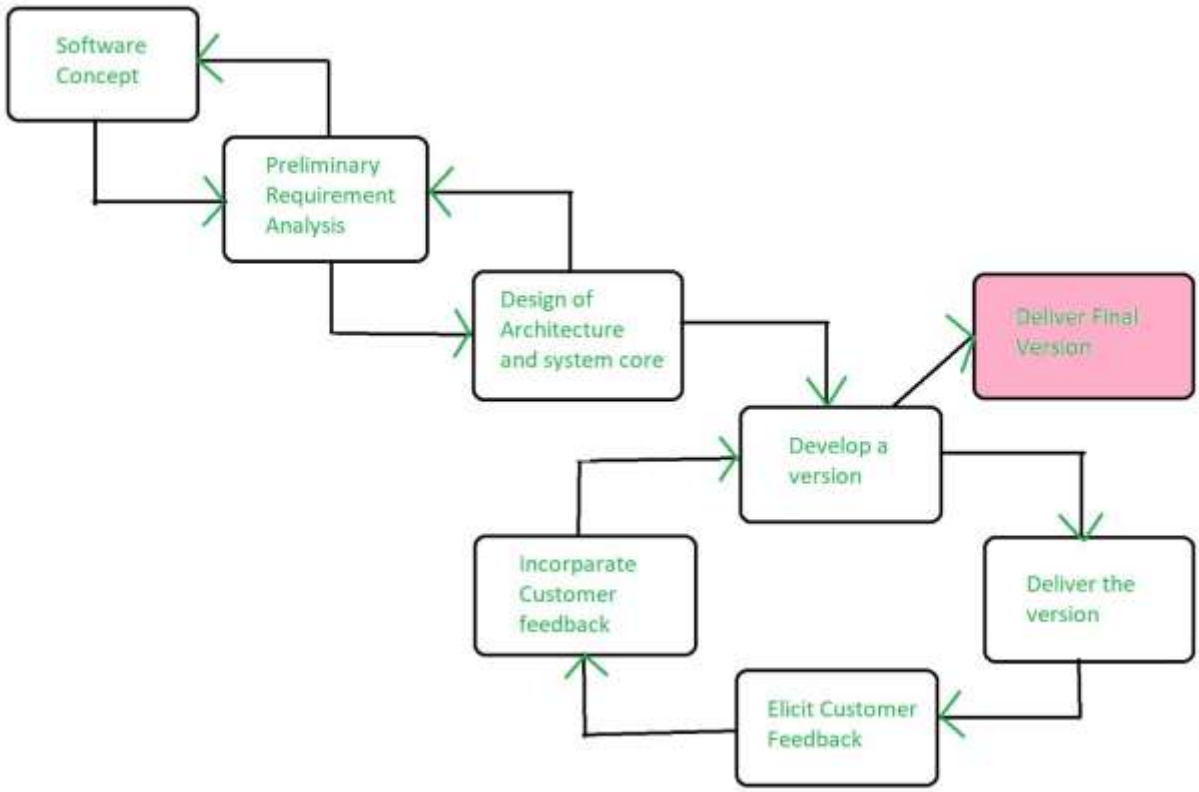


Fig: Evolutionary Model

Application of Evolutionary Model

1. It is used in large projects where you can easily find modules for incremental implementation. Evolutionary model is commonly used when the customer wants to start using the core features instead of waiting for the full software.
2. Evolutionary model is also used in object- oriented software development because the system can be easily portioned into units in terms of objects.

Necessary Conditions for Implementing this Model

1. Customer needs are clear and been explained in deep to the developer team.
2. There might be small changes required in separate parts but not a major change.
3. As it requires time, so there must be some time left for the market constraints.
4. Risk is high and continuous targets to achieve and report to customer repeatedly.
5. It is used when working on a technology is new and requires time to learn.

Advantages Evolutionary Model

1. **Adaptability to Changing Requirements:** Evolutionary models work effectively in projects when the requirements are ambiguous or change often. They support adjustments and flexibility along the course of development.
2. **Early and Gradual Distribution:** Functional components or prototypes can be delivered early thanks to incremental development. Faster user satisfaction and feedback may result from this.
3. **User Commentary and Involvement:** Evolutionary models place a strong emphasis on ongoing user input and participation. This guarantees that the software offered closely matches the needs and expectations of the user.
4. **Improved Handling of Difficult Projects:** Big, complex tasks can be effectively managed with the help of evolutionary models. The development

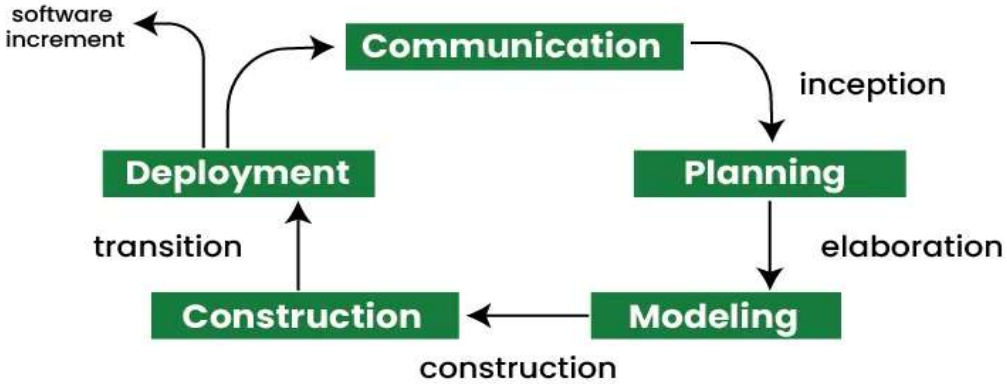
process is made simpler by segmenting the project into smaller, easier-to-manage portions.

Disadvantages Evolutionary Model

1. **Communication Difficulties:** Evolutionary models require constant cooperation and communication. The strategy may be less effective if there are gaps in communication or if team members are spread out geographically.
2. **Dependence on an Expert Group:** A knowledgeable and experienced group that can quickly adjust to changes is needed for evolutionary models. Teams lacking experience may find it difficult to handle these model's dynamic nature.
3. **Increasing Management Complexity:** Complexity can be introduced by organizing and managing several increments or iterations, particularly in large projects. In order to guarantee integration and synchronization, good project management is needed.
4. **Greater Initial Expenditure:** As evolutionary models necessitate continual testing, user feedback and prototyping, they may come with a greater starting cost. This may be a problem for projects that have limited funding.

- **Unified process models**

The Unified Process (UP) in Object-Oriented Analysis and Design (OOAD) is a flexible and iterative approach to developing software. **It focuses on creating working software increments, collaborating with team members, and adapting to changes.**



The Unified Process (UP) in Object-Oriented Analysis and Design (OOAD) is a software development methodology that emphasizes iterative development, collaboration, and flexibility. It is based on the Unified Modeling Language (UML) and is characterized by its use of use cases to drive development, its focus on architecture-centric development, and its emphasis on risk management and incremental delivery. UP is a flexible and adaptable process that can be tailored to meet the specific needs of a project or organization, making it a popular choice for many software development teams.

Importance of Unified Process

- Complex software projects are made more manageable by Unified Process. It breaks them into smaller, iterative chunks.
- Clear guidelines and workflows from Unified Process boost communication. It ensures stakeholder collaboration is seamless.
- Continuous feedback is emphasized by UP's approach. High-quality software meeting requirements are the result.

Key Principles of Unified Process

Below are the key principles of the Unified Process:

- **Iterative and Incremental:** Unified Process divides the development process into multiple iterations, with each iteration adding new functionality incrementally.
- **Use Case Driven:** The Unified Process focuses on identifying and prioritizing use cases that represent the system's functionality from the user's perspective.
- **Architecture-Centric:** The Unified Process emphasizes defining and refining the system architecture throughout the development process.
- **Risk Management:** Unified Process identifies and manages project risks proactively to minimize their impact on the project's success.
- **Continuous Validation:** Unified Process ensures continuous validation of the system's requirements, design, and implementation through reviews, testing, and feedback.

Phases of Unified Process

Unified Process (UP) is characterized by its iterative and incremental approach to software development. The phases in Unified Process provide a structured framework for managing the various activities and tasks involved in building a software system. Here's an in-depth look at each phase:

1. **Inception**

This is the initial phase where the project's scope, objectives, and feasibility are determined. Key activities in this phase include identifying stakeholders, defining the initial requirements, outlining the project plan, and assessing risks. The goal of this phase is to establish a solid foundation for the project and ensure that it is worth pursuing.

2. **Elaboration**

In this phase, the project requirements are analyzed in more detail, and the architecture of the system is defined. Key activities include developing use cases, creating the architectural baseline, identifying key components, and refining the project plan. The goal of this phase is to mitigate major risks and establish a solid architectural foundation for the project.

3. **Construction**
This is the phase where the actual implementation of the system takes place. Key activities include developing, testing, and integrating the system components, as well as continuously verifying that the system meets the requirements. The goal of this phase is to build a complete, high-quality software product that is ready for deployment.

4. **Transition**
In this final phase, the software is deployed to end users. Key activities include user training, final system testing, and transitioning the system to the operations and maintenance team. The goal of this phase is to ensure a smooth transition from development to production and to address any issues that arise during deployment.
These phases are iterative, meaning that they may be revisited multiple times throughout the project to incorporate feedback, make improvements, and address changes in requirements. This iterative approach allows for flexibility and adaptability, making the Unified Process well-suited for complex and evolving software projects.

Roles and Responsibilities of Unified Process
In the Unified Process (UP), there are several roles with specific responsibilities that contribute to the successful development of a software system. These roles and their responsibilities include:

- **Project Manager:** Responsible for overall project planning, scheduling, resource allocation, and coordination of activities. The project manager ensures that the project is completed on time, within budget, and meets the specified requirements.
- **Architect:** Responsible for defining the overall architecture of the system, including its structure, components, and interactions. The architect ensures that the system is scalable, maintainable, and meets the desired quality attributes.
- **Analyst:** Responsible for gathering and analyzing requirements from stakeholders and translating them into detailed specifications. The analyst also helps to identify risks and propose mitigation strategies.
- **Designer:** Responsible for creating detailed designs for the system, including class diagrams, sequence diagrams, and other UML diagrams. The designer works closely with the architect to ensure that the design aligns with the overall architecture.
- **Developer:** Responsible for implementing the system according to the design specifications. The developer writes code, tests it, and integrates it with other components of the system.
- **Tester:** Responsible for testing the system to ensure that it meets the specified requirements and is free of defects. The tester develops test cases, executes them, and reports any issues found.

References:
• Software Engineering by Ian Somerville, Pearson Publication
• https://www.geeksforgeeks.org/